Homework 4 Analysis of 3 Way Implication

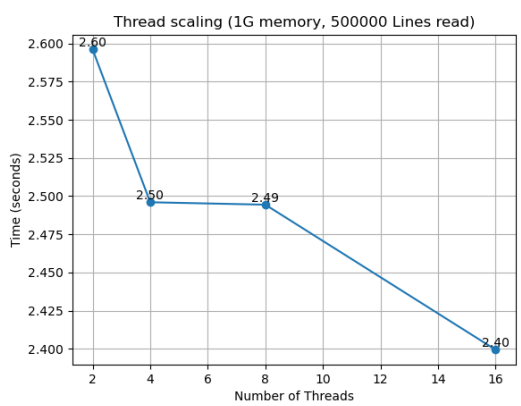
By: Clayton Burgess, Johnny Hug, Samuel Green

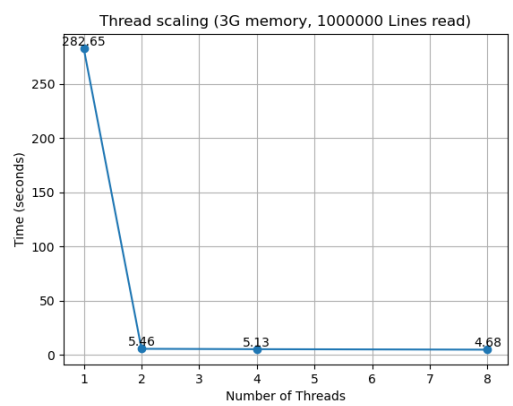
1. Introduction

The following describes the differences between three implementations to find the max ASCII value per line for a specified number of lines read. For all tests in all sections each test was run 5 times and the average of all of those were taken and used as points on the graph as visualizations. Not every test was run exactly 5 times as sometimes the queue was filled with other jobs causing big influxes in numbers. For that purpose, those numbers were considered outliers and dropped to get accurate data. The lines read where from a big file of Wikipedia links.

2. PThreads

PThreads use shared memory from a single machine. So, in terms of using it, on a supercomputer only one process and not a whole cluster of processes are used. Only using one process means that only one node can be used. To test the speed of pthreads on different variables the following tests were ran: changing the number of threads, amount of memory being used, and how many lines are read from our huge file. The first test was how the number of threads affects the performance of the system. One test was run with 1GB of RAM and 500 thousand lines were read from the file. The second test was run using 3GB of RAM on 1 million lines as shown by the first two graphs below.

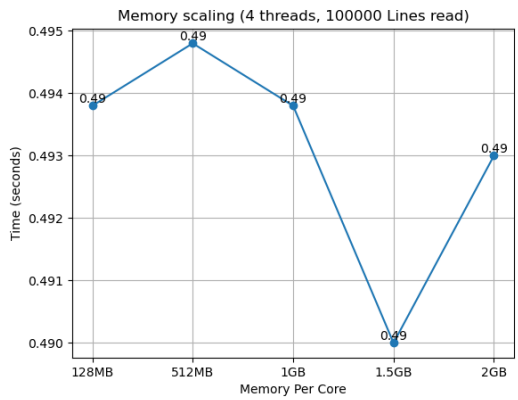


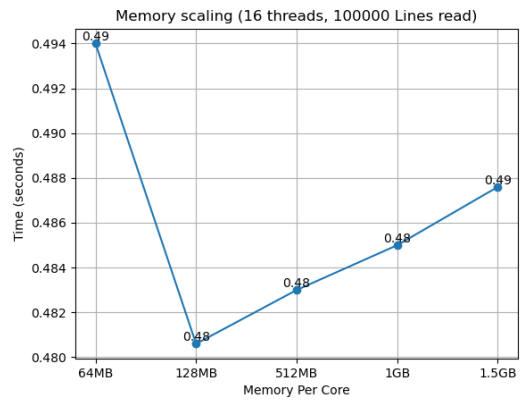




As you can see from the two graphs the one on the left is missing thread number 1, and the right is missing thread number 16. One thread will not work on that set of variables because there is not enough memory to process the lines. Same sort of thing for 16 threads except this exceeds the 1G memory easily. When using 1G the results trend downward throughout the whole range of threads. When we used 3G of data using one thread took minutes compared to the others that took just a few seconds and decreased slower as the number of threads increased.

For the next test that was run we changed the amount of memory that was used to run a certain number of threads with a certain number of lines read from the file. One was 4 threads and we processed 100 thousand lines, and the other we used 16 threads on 100

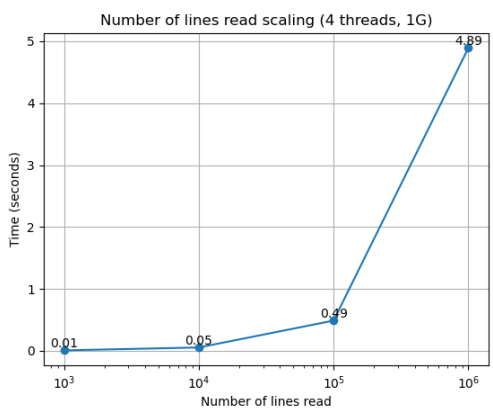
thousand lines again. Here are the results on the two graphs below.

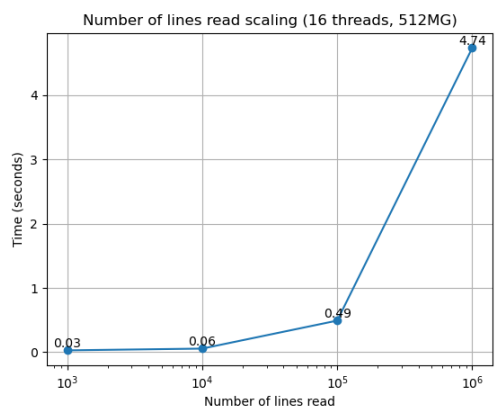



We see kind of the same trend as we did with the threads. The fewer number of threads used the lower amount of memory performed weaker than the higher amount of memory and vice versa for the higher thread. 64MB was not enough memory to process the number of rows with 4 threads and 2G with 16 threads easily went over the amount of memory we were trying to use. We also see that one is significantly lower than rest in both cases which doesn't necessarily mean that amount of memory is optimal, there might have just been more noise for that group of runs.

The last set of tests that were performed was on how the systems react to reading different numbers of lines from the file. The constant variables for these were 4 threads on 1G and 16 threads on 512MB. Results shown below.



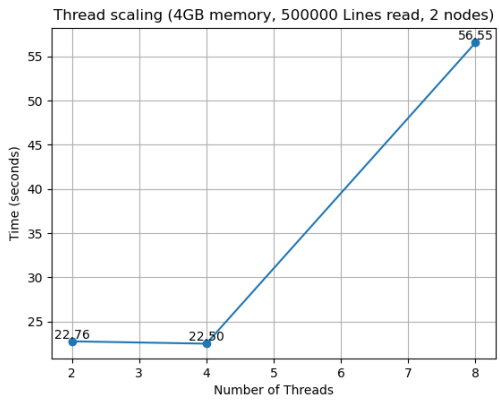

These perform very similarly and how one might expect. The greater number of rows read the more time it takes to complete the task. If we boil it down further in most instances, reading with more threads and less memory performs a tiny bit slower.

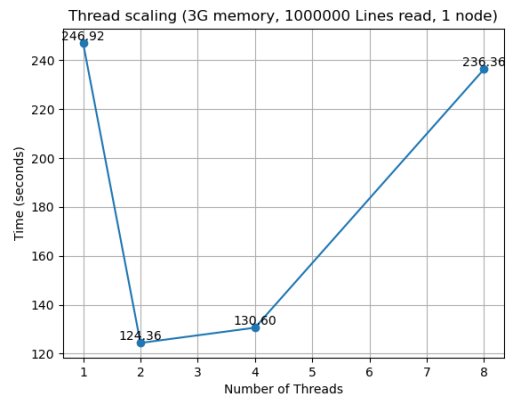
To conclude this section on Pthreads we can see that the performance between each set of tests has its similarities. However, there are a few other aspects that come into play. We can use different networks to run these tests. In the testing performed here there were no major differences noticed. On the other hand, what was noticed is that when there was a peak in jobs being run on the system there was a fair number of inconsistencies with the data. Too big to be considered for the averages shown in the

graphs as the times were jumping all around. For that purpose, those times were considered outliers and dropped for the purpose of having the most consistent data.

3. MPI

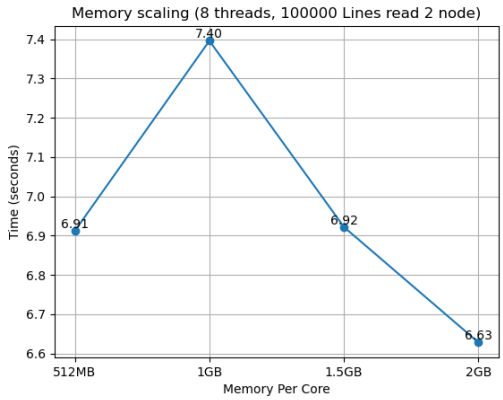
MPI splits work between multiple machines to get large amounts of work done without overwhelming just one machine. This causes more overhead for smaller jobs causing them to be worse in some cases. The first test ran was on the thread count. One test was run with 4GB of memory reading 500 thousand lines, the second used 3GB of memory on 1 million lines of data.

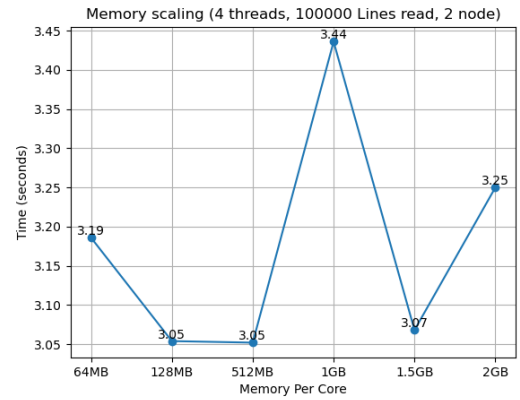



These graphs show a healthy amount of change in the times taken to run the number of lines read in. In both cases tests with 16 threads were attempted to be run but unfortunately was not a configuration. The graph on the left is missing thread count 1. This is because through this process if the number of nodes used was greater than the number of threads the test was unable to run. This is more than likely due to not being able to split 1 thread into half thread and so on with other tests not being able to split the data or between the nodes evenly. Another difference is the upward trend started at 1 thread on the graph on the right. This could possibly be due to more jobs being run at the time these tests were done.

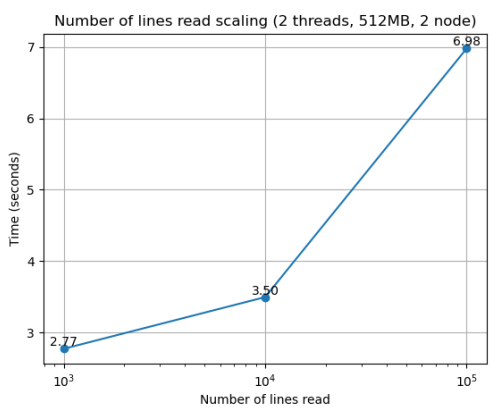
The next text was the memory test. The main problem with this one is the same as we saw in pthreads. That being that there was not enough memory available to spread across the threads and the nodes selected for these tests. The results are in the graphs below.

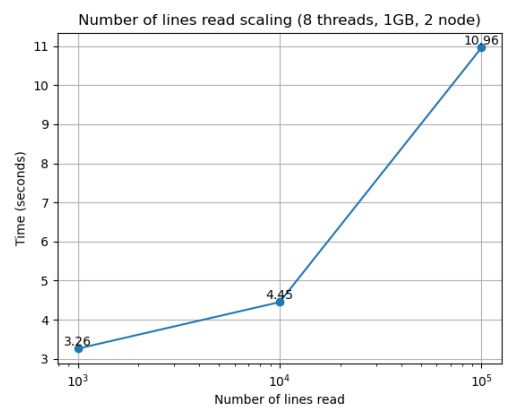



One test was run with 8 threads, read 100 thousand lines with 2 nodes, while the other was ran with 4 threads, read 10 thousand lines also on 2 nodes. Like the number of threads, the graph on the left is missing 64MB and 128MB because there was not enough memory to perform the test with those parameters. The graphs are sporadic jumping around between test points. This is probably due to noise or an increase in the jobs being submitted when these tests were being run. Now let us examine the last test that was some between pthreads and MPI.

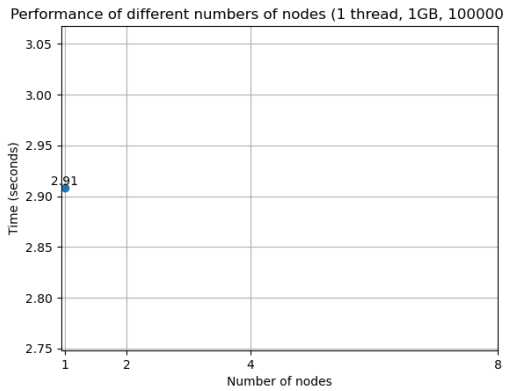
Next, we examine how the number of lines read affects the time it takes to process using MPI. The results are below. We ran tests with 2 threads, 512MB of data and 2 nodes and one with 8 threads, 1GB of data, and 2 nodes.

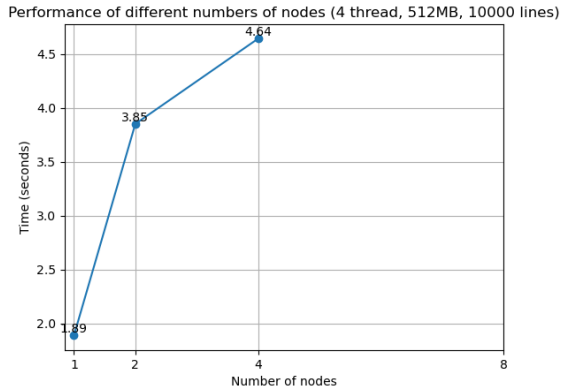



These two tests ran very similarly. When you add more lines to be read the time increases. The tests using 2 threads overall were fewer because there did not need to be as many resources split to complete the task. The test with 1 million lines in both cases there was not enough memory to process the number of lines that were being asked.

The last test we ran only applied to MPI as that is the only one we can change how many machines the tests are run on to split up the work.



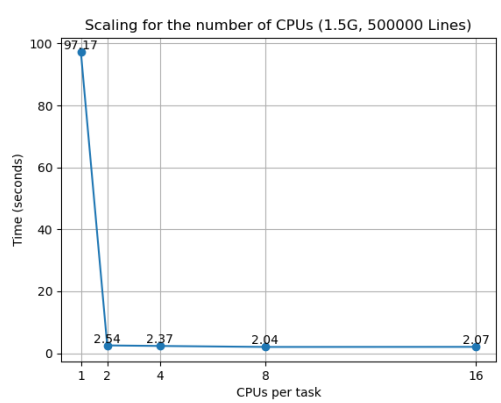



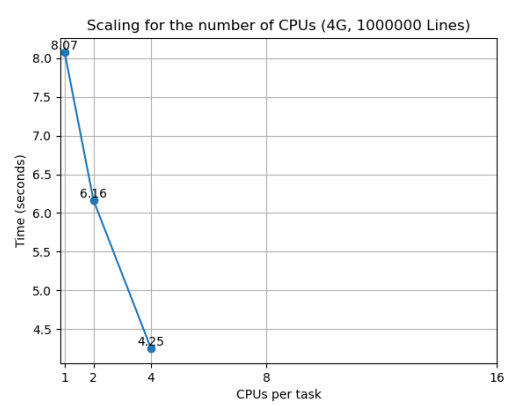

As seen above we can come to the same conclusion as we did with the threads test. The test on the left only shows one point at node number using 1 thread for all the tests. There is only one point because after the number of nodes excessed the number of nodes there is not a good or even way to split the data. The same can be said about the graph of the right except this time we used 4 threads, so the tests ran tell the nodes equaled 4 then failed for the remaining node counts.

Overall MPI ran slower than pthreads. This doesn't necessarily mean it is slower all the time but in this instance it was. The belief of this is that the number of lines read in during these tests were too small for MPI to make a difference. Because the sharing and splitting of data between nodes overhead and keeping track of everything transferring it between nodes took longer than just having one node do the all the work as in pthreads.

4. OpenMP

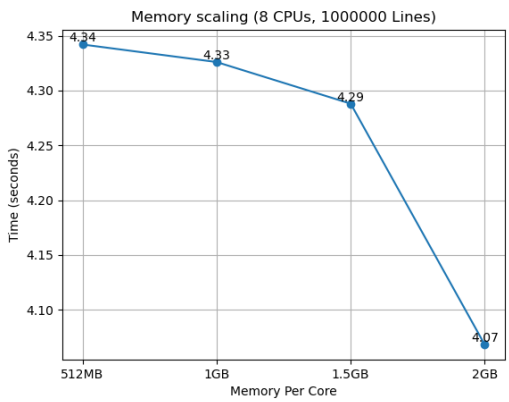
OpenMP is the last implementation that was tested. It is a lot like pthreads but a bit simpler sharing data between one node but it doesn't have to manipulate the arrays like the following two implementations did. That also means that the number of nodes for these tests was left at 1 to ensure only 1 machine was used to get the time. The first test was run on how the number of CPUs used affected the time it took to scan a certain number of lines.

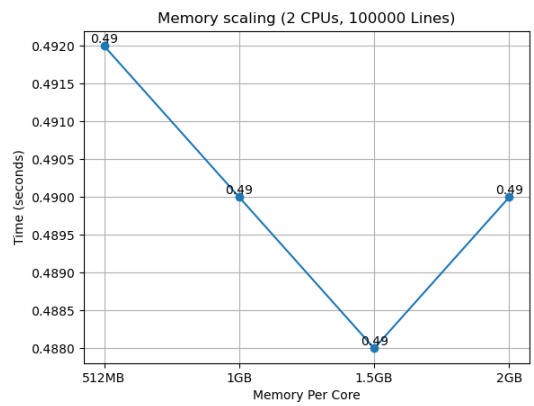


These tests ran like the pthreads threads tests did. With it taking the most time using the least number and less time when we reached higher numbers. Suggesting that they run the same, and they do because they don't share the data between multiple nodes. The graph on the right is missing 8, and 16 CPUs. This is because that configuration was not available on beocat. However, it still has the same trend as the other graph with a downward trend in time as the numbers become greater.

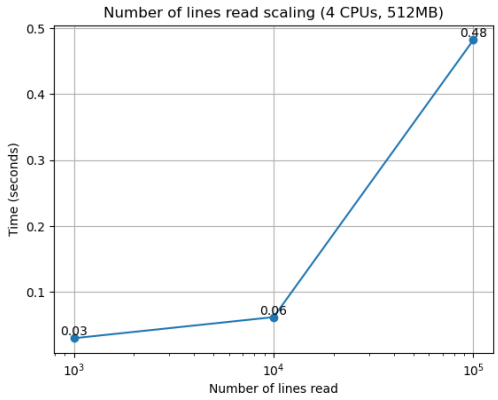
The next tests ran was the memory test. For this test the numbers stayed right around the same range. That falls in line with the other memory tests as well even though the lines in the graphs of all of them are all over the place. The results of the OpenMP memory test are below.

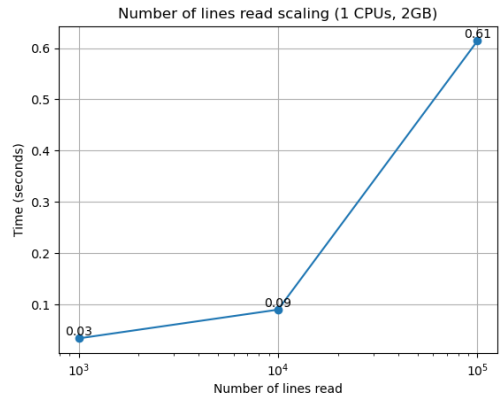



This suggests for all the implementations that memory doesn't really affect the running time. It's just the running time of the implementation that makes these numbers seem bigger or smaller between all the implementations. This similar value between all of them seems to say that if there is enough memory to perform the work the numbers will be around the same. The only difference would be the noise in the background as to why these are not straight lines all the way across even with the graphs blown up in smaller margins.

The last test that was ran was the number of lines read. Just like the memory tests the results are similar in all three implementations.



In all of them the number of lines increased the longer it took to run. This makes sense as with more lines comes more computation and manipulation to get through the increase of lines. In all tests in this OpenMP section if not all the data points were present there either not enough memory was to run or the configuration was not available.

5. Conclusion

Each implementation had its strengths and weaknesses. For Pthreads it was efficient as it only used one node to run the data through. If we used a bigger number of lines to process the MPI would have come closer to being like pthreads. The downside of pthreads comes when we compare it with OpenMP. As OpenMP does not manipulate the arrays as much as pthreads causing it to be just a tad slower on similar tests. MPI is great for big jobs that pthreads and OpenMP can not handle. However, for these tests it performed the worst. Trying to create similar tests for all implementations to show the outputs. If the jobs were bigger and required more work MPI would have possibly shined but with the extra overhead and having to keep track of what data is where and combining it all back together again at the end lead to greater performance numbers. OpenMP would perform the test as shown in the graphs. The performance times are lower as stated above the array manipulation was much smaller per row than the other implementations. The downside was the inability to run all the tests. Sure, all implementations couldn't run some of the tests depending on parameters but OpenMP was the one that refused the most.